

Mobile Devices Programming
Practical Assignment, Winter Semester 2026/2027

Delivery date: December 12, 2026

This document specifies the requirements for this course's practical assignment. Your task is to implement the **DroidMentor** application, a Bring Your Own Key (BYOK) Large Language Model (LLM) client that acts as a senior Android development expert¹. Communication with the LLM is implemented using Gemini's REST API, through the [generateContent endpoint](#).

The application relies on modern Android development practices. For dependency management, the application must use the Application class as a service locator (the use of Hilt/Dagger is not permitted). Network communication is strictly handled via the Ktor client and Kotlinx Serialization libraries. Local storage of chat histories is managed by the Room persistence library, while sensitive user credentials (the BYOK API key) are stored securely using DataStore.

A core requirement of **DroidMentor** is the use of the stateless variant of Gemini's API. Students are not allowed to use existing server-side multi-turn session IDs. Instead, the client application is responsible for managing all conversational state. To maintain the context of each chat session, the application must construct the request payload with the entire conversation history, alternating user and model roles, in each new network interaction. The model must also be configured via `system_instruction` to adopt the persona of an Android engineering mentor, providing sound advice on Android development best practices.

The **DroidMentor** application is designed as an **offline-first** experience. All conversations must be accessible without network connectivity. Network availability must be verified prior to initiating any new API calls; if offline, the application must clearly convey to the user that new model interactions are unavailable at the moment. Furthermore, as the application relies on an external REST API, the application must gracefully handle HTTP error responses (e.g., HTTP 429 Too Many Requests or HTTP 500 Internal Server Error), ensuring it remains stable and provides appropriate error feedback to the user.

The **DroidMentor** application contains the following screens:

- Title - main menu;
- Chat History - lists previously saved conversations;
- Active Chat - displays the selected conversation;
- About - displays information regarding the application and its authors;
- Settings - used to configure the Gemini API key secret

The suggested application navigation is depicted in Figure 1, where arrows represent navigation options. Usage starts at the Title screen, where the application's main menu is presented. There, the user can either elect to see the list of past conversations (Chat History screen), view or change the application settings (Settings screen) or consult the available application information (About screen). From the Chat History screen, the user can navigate to the screen used to engage in conversation with the mentor (Active Chat screen), either by selecting a previous conversation, or by starting a new one. When the application is launched, if the user was previously engaged in a conversation, the user is automatically directed to the

¹The mentor's expertise depends entirely on the model's pre-existing training data. You are only required to set the persona using a basic `system_instruction`. You do not need to implement Retrieval-Augmented Generation (RAG) architectures or experiment with advanced prompt engineering techniques.

Active Chat screen. A user is considered previously engaged if they did not explicitly close or navigate away from the active conversation before the application was terminated.

Authors are free to improve on the suggested navigation experience. The remaining details, such as the general appearance of the UI, which orientation is used in each screen, and further interaction details are left to the authors' discretion. A good reference for designing user experiences (UX) is ["Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability", by Steve Krug](#)

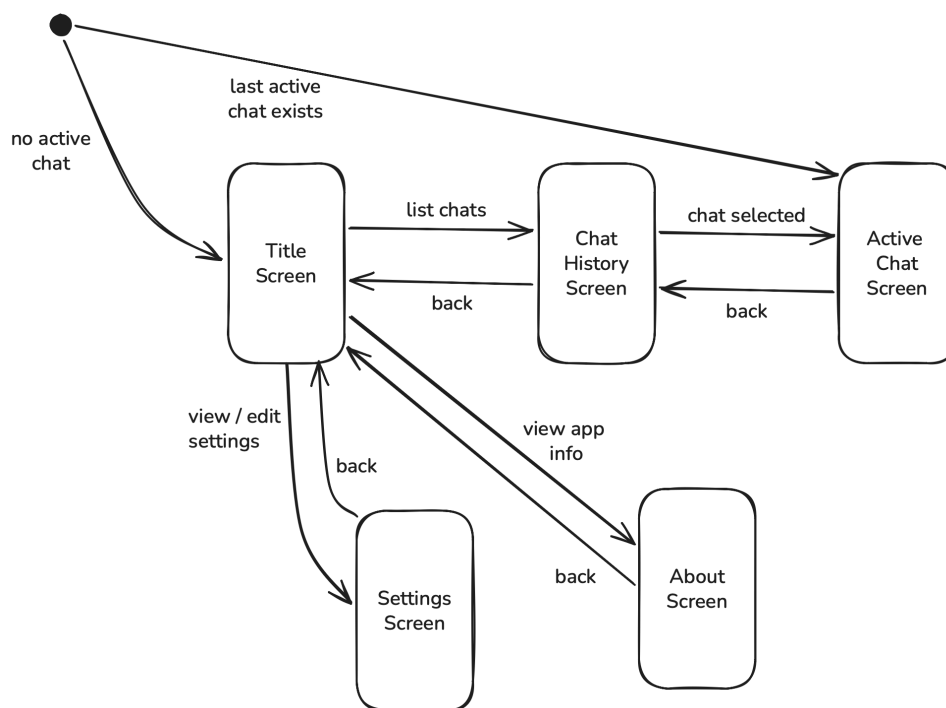


Figure 1 - Suggested application navigation

Requirements

In addition to the previously described requirements, the application must also support the removal of previous conversations and conversation rewrite, meaning, the user may elect to edit a message from a conversation, regardless of its position. Naturally, all subsequent exchanges become invalid and are to be replaced by the new ones.

As an optional (and valued) requirement, the conversation can include images either obtained from the device's image gallery or captured by its camera. Consider [this Gemini API endpoint](#) for that purpose. If this optional requirement is implemented, the application must store the image files in the device's local storage and persist only the file URI or file path in the Room database. Storing raw image data (such as Base64 encoded strings) directly in the database is strictly prohibited.

The project's milestones are presented next. In each milestone, students are expected to deliver what was planned. Deliveries are carried out by creating "mentor_X" tags, where "X" stands for the milestone number, in the group's GitHub repository. The repository is created by each group independently. Full access must be granted to the course's professors, [Prof. Paulo Pereira](#) and [Prof. Diogo Cardoso](#). Each repository **must contain at its root the README.md file with the identification of the group's members.**

Milestone 1 (week 3 - 21/09/2026)

Deliverables: The project plan identifying the deliverables of each of the project's milestones. You may want to check the [course's plan](#) with the order by which each subject will be presented.

Acceptance criteria:

- The repository is correctly tagged (mentor_1) and contains the project plan timeline indicating which tasks are planned and to whom they are assigned
- The README.md file contains the identification of the group's members

Milestone 2 (week 6 - 12/10/2026)

Deliverables: Presentation of the current state of the project using a video (5–7 minutes) containing

- Demonstration of the implemented functionalities
- Discussion of the most relevant decisions
- Description of the elected verification strategy
- Current status of the project

Acceptance criteria:

- The repository is correctly tagged (mentor_2) and contains the video link in the README.md

Milestone 3 (week 11 - 16/11/2026)

Deliverables: Presentation of the current state of the project using a video (5–7 minutes) containing

- Demonstration of the implemented functionalities
- Discussion of the most relevant decisions
- Current status of the project

Acceptance criteria:

- The repository is correctly tagged (mentor_3) and contains the video link in the README.md

Final Milestone (Deadline 12/12/2026)**Final Demo**

- Deliver a complete and fully functional version of the application
- All features from previous milestones must be integrated and working together
- The group must prepare a short demo video (5–7 minutes) showcasing the application in action
- The video link must be included in the repository's README.md file

Acceptance criteria:

- The app is complete and stable, with all planned screens accessible
- All functionality works as specified in earlier milestones
- The demo video clearly shows the application's features
- The repository is correctly tagged (mentor_f) and contains the video link in the README.md

Due date

December 12, 2026

ISEL, September 16, 2026